

# COMP3308/3608 Week 8 - Introduction to Neural Networks, Multilayer Perceptrons, and Backpropagation

**Author**

K. Yu

2026

## Contents

|          |                                                                      |          |
|----------|----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction to Neural Networks</b>                               | <b>2</b> |
| 1.1      | Artificial Neural Networks . . . . .                                 | 2        |
| 1.2      | Artificial Neuron Model . . . . .                                    | 2        |
| 1.3      | Taxonomy of Neural Networks . . . . .                                | 3        |
| 1.4      | Perceptron . . . . .                                                 | 4        |
| 1.5      | Perceptron Learning Rule . . . . .                                   | 5        |
| 1.6      | Epochs and stopping criterion . . . . .                              | 6        |
| 1.7      | Perceptron convergence and limitations . . . . .                     | 6        |
| <b>2</b> | <b>Multilayer Perceptron</b>                                         | <b>7</b> |
| 2.1      | Motivation . . . . .                                                 | 7        |
| 2.2      | Architecture . . . . .                                               | 7        |
| 2.3      | Activation Functions . . . . .                                       | 7        |
| 2.3.1    | Hyperbolic Tangent Sigmoid . . . . .                                 | 8        |
| 2.4      | Loss Function and Gradient Descent . . . . .                         | 9        |
| 2.5      | Backpropagation for one hidden layer . . . . .                       | 10       |
| 2.6      | Backpropagation Algorithm . . . . .                                  | 13       |
| 2.7      | Momentum and Learning Rate . . . . .                                 | 14       |
| 2.7.1    | Momentum . . . . .                                                   | 14       |
| 2.8      | Overfitting and Underfitting . . . . .                               | 15       |
| 2.9      | Capabilities and Limitations of Multilayer Neural Networks . . . . . | 16       |

# 1 Introduction to Neural Networks

## 1.1 Artificial Neural Networks

An **Artificial Neural Network** (ANN) is a network of artificial neurons. It was inspired by biological neural systems, but artificial neurons are only very simple abstractions of biological neurons.

A neural network consists of:

- **neurons**, also called units or nodes;
- **weighted connections** between neurons;
- **input**, **hidden**, and **output** layers;
- a **learning rule** for modifying weights.

A neural network represents a transformation from inputs to outputs  $f(\mathbf{x}; \mathbf{w}, b)$ . By changing the weights and biases, the network changes the function it represents.

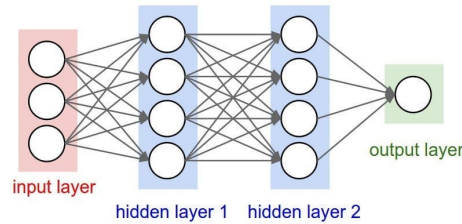


Figure 1: A simple neural network diagram.

## 1.2 Artificial Neuron Model

### Weighted sum

For one neuron, let the input be  $\mathbf{x} \in \mathbb{R}^d$ . The neuron has weights  $\mathbf{w} \in \mathbb{R}^d$  and bias  $b \in \mathbb{R}$ . The neuron first computes a weighted sum

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{j=1}^d w_j x_j + b.$$

Then it applies an activation function  $g$ :

$$a = g(z) = g(\mathbf{w}^\top \mathbf{x} + b).$$

Here,  $\mathbf{x}$  comes from the data,  $\mathbf{w}$  and  $b$  are learned parameters, and  $a$  is the neuron output.

### Bias term

The bias  $b$  can be seen as a special weight connected to a constant input 1. So  $z = \mathbf{w}^\top \mathbf{x} + b$  can also be written as

$$z = w_1 x_1 + \cdots w_d x_d + b \cdot 1.$$

The bias shifts the decision boundary. Without a bias, the decision boundary must pass through the origin.

## 1.3 Taxonomy of Neural Networks

### Feedforward and recurrent networks

Neural networks can be classified by their connection structure.

- A **feedforward neural network** (FFN) has no cycles. Information flows from input layer to output layer.
- A **recurrent neural network** (RNN) has cycles or feedback connections. It is useful for temporal or sequential data.

Examples include:

- recurrent backpropagation networks;
- associative memory networks;
- LSTM networks.

### Supervised and unsupervised networks

Neural networks can also be classified by learning type. A **supervised neural network** is trained using labelled examples  $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ . It is usually used for classification or regression.

Examples include:

- perceptrons;
- ADALINE;
- multilayer backpropagation networks;
- radial basis function networks;
- deep neural networks.

An **unsupervised neural network** learns patterns without labels.

Examples include:

- Hebbian networks;
- competitive networks;
- self-organising maps.

## 1.4 Perceptron

A **perceptron** is a simple supervised neural network. It has:

- input features;
- one output neuron;
- a step activation function;
- binary output.

For binary classification, we commonly use labels  $y^{(i)} \in \{0, 1\}$ . The perceptron computes

$$z = \mathbf{w}^\top \mathbf{x} + b.$$

Then it applies a step function

$$a = \text{step}(z),$$

where

$$\text{step}(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ 0 & \text{if } z < 0. \end{cases}$$

So the perceptron prediction is  $\hat{y} = a$ .

### Perceptron as a classifier

The perceptron divides the input space into two regions. The decision boundary is  $\mathbf{w}^\top \mathbf{x} + b = 0$ .

- If  $\mathbf{w}^\top \mathbf{x} + b \geq 0$ , then  $\hat{y} = 1$ .
- If  $\mathbf{w}^\top \mathbf{x} + b < 0$ , then  $\hat{y} = 0$ .

Therefore, a perceptron forms a **linear decision boundary**. In  $\mathbb{R}^2$ , this boundary is a line. In  $\mathbb{R}^d$ , this boundary is a hyperplane.

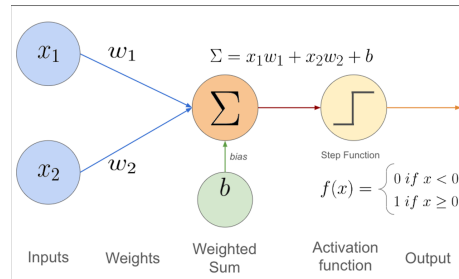


Figure 2: A perceptron with input  $\mathbf{x} = [x_1 \ x_2]^\top$ .

**Example 1.4.1** (Two-dimensional example). Suppose  $d = 2$ . Let  $w_1 = 1$ ,  $w_2 = 1$ , and  $b = -1$ . Then the decision boundary is:

$$x_1 + x_2 - 1 = 0.$$

Equivalently,

$$x_2 = 1 - x_1.$$

The intercepts are  $x_1 = 0 \implies x_2 = 1$  and  $x_2 = 0 \implies x_1 = 1$ . So the boundary is a straight line through  $(0, 1)$  and  $(1, 0)$ .

## 1.5 Perceptron Learning Rule

### Training setup

We are given a labelled training set  $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ ,  $\mathbf{x}^{(i)} \in \mathbb{R}^d$ ,  $y^{(i)} \in \{0, 1\}$ . The goal is to learn (estimate) weights  $\mathbf{w}$  and bias  $b$  such that the perceptron correctly classifies the training examples. The perceptron starts with random or small initial weights, then updates them example by example.

### Output error

For one training example  $(\mathbf{x}^{(i)}, y^{(i)})$ , compute  $a^{(i)} = \text{step}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$ . The error is

$$e^{(i)} = y^{(i)} - a^{(i)}.$$

Since  $y^{(i)}, a^{(i)} \in \{0, 1\}$ , the error can be  $e^{(i)} \in \{-1, 0, 1\}$ .

Interpretation:

- $e^{(i)} = 0$ : prediction is correct;
- $e^{(i)} = 1$ : target is 1 but output is 0;
- $e^{(i)} = -1$ : target is 0 but output is 1.

### Weight update rule

The perceptron learning rule is:

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} + e^{(i)} \mathbf{x}^{(i)}, \\ b &\leftarrow b + e^{(i)}.\end{aligned}$$

If the prediction is correct, then  $e^{(i)} = 0$ , so no update is made. If the target is 1 but the perceptron outputs 0, then  $e^{(i)} = 1$ , so the input vector is added to the weight vector. If the target is 0 but the perceptron outputs 1, then  $e^{(i)} = -1$ , so the input vector is subtracted from the weight vector.

### Perceptron learning algorithm

The training algorithm is given below as pseudocode.

---

**Algorithm 1** Perceptron Learning Algorithm

---

**Require:** training set  $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$

Initialise  $\mathbf{w}$  and  $b$  to small random values.

Repeat for multiple epochs until stopping criterion is satisfied:

```
for  $i = 1 \dots n$  do  
     $a^{(i)} \leftarrow \text{step}(\mathbf{w}^\top \mathbf{x}^{(i)} + b)$   
     $e^{(i)} = y^{(i)} - a^{(i)}$   
     $\mathbf{w} \leftarrow \mathbf{w} + e^{(i)} \mathbf{x}^{(i)}$   
     $b \leftarrow b + e^{(i)}$   
end for
```

---

## 1.6 Epochs and stopping criterion

An **epoch** is one complete pass through the training set. That means each training example is presented once to the perceptron, and the weights may be updated.

If the training set is  $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$ , then one epoch processes all  $n$  examples once.

At the end of each epoch, we check whether all training examples are correctly classified. To check this, we pass all examples through the perceptron again and compare predictions with targets.

Training stops when either:

1. all training examples are correctly classified; or
2. a maximum number of epochs is reached.

## 1.7 Perceptron convergence and limitations

**Theorem 1.7.1** (Perceptron convergence theorem). *If the training examples are linearly separable, i.e. there exists a hyperplane  $\mathbf{w}^\top \mathbf{x} + b = 0$  that correctly separates the two classes, then the perceptron learning rule is guaranteed to converge to a solution in a finite number of steps.*

### Limitation

The perceptron can only learn linear decision boundaries. Therefore, it cannot solve problems that are not linearly separable. A classic example is XOR.

The perceptron also does not search for the "best" separating boundary. It simply stops once it finds a boundary that correctly classifies the training data.

## 2 Multilayer Perceptron

### 2.1 Motivation

Although a single perceptron can only create one linear decision boundary, a multilayer network can combine many such boundaries.

A multilayer perceptron (MLP) contains:

input layer  $\rightarrow$  hidden layer(s)  $\rightarrow$  output layer.

The hidden neurons learn intermediate representations, and the output layer combines them to produce the final prediction.

### 2.2 Architecture

A standard MLP is usually feedforward, fully connected, and supervised. For numerical data, there is usually one input neuron per attribute. For categorical data, one-hot encoding is often used. For classification, there is often one output neuron per class.

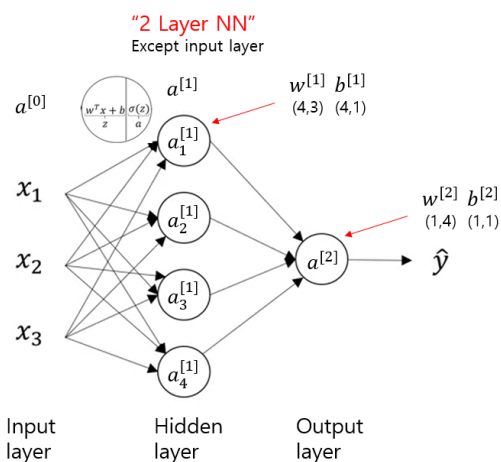


Figure 3: A 2-layer MLP.

### 2.3 Activation Functions

For backpropagation, the activation function must be differentiable, because weight updates are derived using gradients. A neuron computes

$$a = f(\mathbf{w}^\top \mathbf{x} + b).$$

Common activation functions include sigmoid, tanh, and ReLU.

## Sigmoid

The sigmoid function is given by

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Its output is in the interval  $(0, 1)$ . The derivative is

**Lemma 2.3.1.** *The derivative of the sigmoid function  $\sigma(x) = \frac{1}{1+e^{-x}}$  satisfies*

$$\sigma'(x) = \sigma(x)(1 - \sigma(x)).$$

*Proof.*

$$\begin{aligned}\sigma'(x) &= \frac{d}{dx} \frac{1}{1 + e^{-x}} \\ &= -\frac{1}{(1 + e^{-x})^2} \frac{d}{dx} (1 + e^{-x}) \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} \\ &= \frac{1}{1 + e^{-x}} \left( 1 - \frac{1}{1 + e^{-x}} \right) \\ &= \sigma(x)(1 - \sigma(x)).\end{aligned}$$

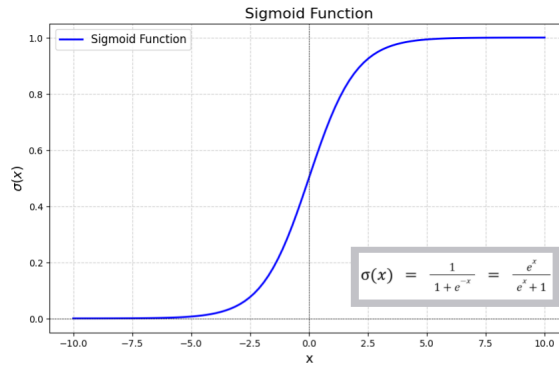


Figure 4: The plot of sigmoid function.

### 2.3.1 Hyperbolic Tangent Sigmoid

The hyperbolic tangent sigmoid is

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}.$$

Its output is in the interval  $(-1, 1)$ .

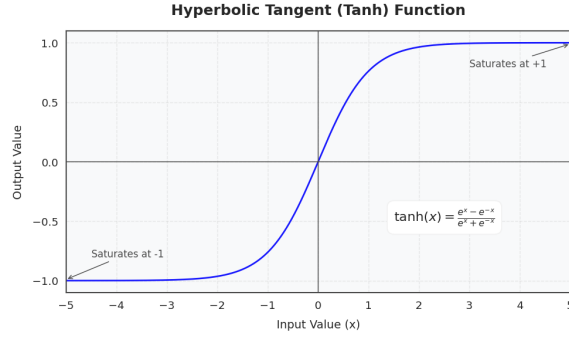


Figure 5: The plot of tanh function.

## 2.4 Loss Function and Gradient Descent

### Loss function

For one training example, suppose the network has output neurons indexed by  $i$ . Let  $d_i$  be the desired target value for output neuron  $i$ , and  $o_i$  be the actual output of output neuron  $i$ . The squared loss is

$$\ell = \frac{1}{2} \sum_i (d_i - o_i)^2.$$

The factor  $1/2$  is used because it cancels the 2 when differentiating. For the full training set,

$$L_{\text{train}} = \sum_{r=1}^n \ell^{(r)} = \frac{1}{2} \sum_{r=1}^n \sum_i \left( d_i^{(r)} - o_i^{(r)} \right)^2.$$

$L$  can be viewed as a function of weights.

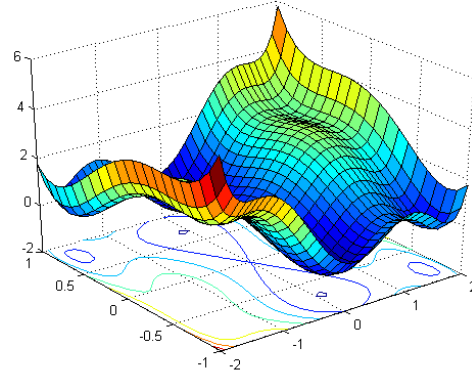


Figure 6: Loss surface in the weight space  $\mathbf{w} = [w_1 \ w_2]^\top$ .

## Gradient descent

The weights are the parameters of the network, training means estimating weights that reduce the loss.

For a weight  $w$ , gradient descent updates it by moving in the negative gradient direction:

$$w \leftarrow w - \eta \frac{\partial L}{\partial w},$$

where  $\eta > 0$  is the learning rate, it controls the step size.

- If  $\eta$  is too small, convergence is slow.
- If  $\eta$  is too large, training may be unstable and diverge.

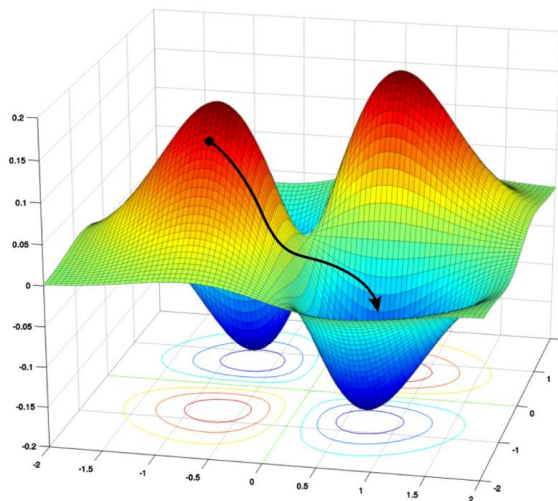


Figure 7: Steepest gradient descent on the loss surface.

## 2.5 Backpropagation for one hidden layer

Consider a network with one hidden layer. Let  $o_k$  be the input value from input neuron  $k$ . For the raw input,  $o_k = x_k$ . For hidden neuron  $j$ ,

$$\text{net}_j = \sum_k w_{kj} o_k + b_j.$$

Its output is

$$o_j = f(\text{net}_j).$$

For output neuron  $i$ ,

$$\text{net}_i = \sum_j w_{ji} o_j + b_i.$$

Its output is

$$o_i = f(\text{net}_i).$$

The loss for one example is

$$\ell = \frac{1}{2} \sum_i (d_i - o_i)^2.$$

### Output layer weight derivation

For a weight  $w_{ji}$  from hidden neuron  $j$  to output neuron  $i$ , gradient descent gives

$$w_{ji} \leftarrow w_{ji} - \eta \frac{\partial \ell}{\partial w_{ji}}.$$

Using chain rule,

$$\frac{\partial \ell}{\partial w_{ji}} = \frac{\partial \ell}{\partial o_i} \frac{\partial o_i}{\partial \text{net}_i} \frac{\partial \text{net}_i}{\partial w_{ji}}.$$

Now compute each term.

$$\frac{\partial \ell}{\partial o_i} = \frac{\partial}{\partial o_i} \frac{1}{2} \sum_i (d_i - o_i)^2 = -(d_i - o_i),$$

$$\frac{\partial o_i}{\partial \text{net}_i} = f'(\text{net}_i),$$

$$\frac{\partial \text{net}_i}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}} \left( \sum_k w_{kj} o_k + b_j \right) = o_j.$$

Therefore,

$$\frac{\partial \ell}{\partial w_{ji}} = -(d_i - o_i) f'(\text{net}_i) o_j.$$

Thus, let

$$\Delta w_{ji} = \eta [-(d_i - o_i) f'(\text{net}_i) o_j] = \eta (d_i - o_i) f'(\text{net}_i) o_j.$$

Define the output neuron error signal

$$\delta_i = (d_i - o_i) f'(\text{net}_i).$$

Then

$$\Delta w_{ji} = \eta \delta_i o_j,$$

and

$$w_{ji} \leftarrow w_{ji} + \eta \delta_i o_j.$$

### Hidden layer weight derivation

For a weight  $w_{kj}$  from input neuron  $k$  to hidden neuron  $j$ ,

$$\Delta_{kj} = -\eta \frac{\partial \ell}{\partial w_{kj}}.$$

The hidden neuron  $j$  affects the loss indirectly through all output neurons connected to it. Therefore,

$$\frac{\partial \ell}{\partial w_{kj}} = \frac{\partial \ell}{\partial o_j} \frac{\partial o_j}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{kj}}.$$

The term  $\frac{\partial \ell}{\partial o_j}$  collects the influence of hidden neuron  $j$  on all output neurons:

$$\frac{\partial \ell}{\partial o_j} = \sum_i \frac{\partial \ell}{\partial \text{net}_i} \frac{\partial \text{net}_i}{\partial o_j}.$$

From the output-layer derivation,

$$\frac{\partial \ell}{\partial \text{net}_i} = -(d_i - o_i) f'(\text{net}_i) = -\delta_i.$$

Also,

$$\text{net}_i = \sum_j w_{ji} o_j + b_i,$$

so

$$\frac{\partial \text{net}_i}{\partial o_j} = w_{ji}.$$

Therefore,

$$\frac{\partial \ell}{\partial o_j} = \sum_i (-\delta_i) w_{ji}.$$

Thus,

$$\frac{\partial \ell}{\partial w_{kj}} = \left[ \sum_i (-\delta_i) w_{ji} \right] f'(\text{net}_j) o_k.$$

So

$$\Delta_{kj} = -\eta \left[ \sum_i (-\delta_i) w_{ji} \right] f'(\text{net}_j) o_k = \eta f'(\text{net}_j) \left( \sum_i \delta_i w_{ji} \right) o_k.$$

Define the hidden neuron error signal

$$\delta_j = f'(\text{net}_j) \sum_i \delta_i w_{ji}.$$

Then

$$\Delta w_{kj} = \eta \delta_j o_k$$

and the update rule is

$$w_{kj} \leftarrow w_{kj} + \eta \delta_j o_k.$$

### General backpropagation rule

For a connection from neuron  $p$  to neuron  $q$ ,

$$\Delta w_{pq} = \eta \delta_q o_p,$$

where  $o_p$  is the output activation of the previous neuron.

For an output neuron  $q$ ,

$$\delta_q = (d_q - o_q) f'(\text{net}_q).$$

For a hidden neuron  $q$ ,

$$\delta_q = f'(\text{net}_q) \sum_r \delta_r w_{qr}$$

where  $r$  ranges over neurons in the next layer.

This is why the algorithm is called backpropagation, the error signal  $\delta$  is computed first at the output layer, then propagated backward through the hidden layers.

## 2.6 Backpropagation Algorithm

### Stochastic/Incremental gradient descent

For each training example  $(\mathbf{x}^{(r)}, y^{(r)})$ :

1. Feed  $\mathbf{x}^{(r)}$  forward through the network.
2. Compute the output  $o_i$ .
3. Compute the loss

$$\ell^{(r)} = \frac{1}{2} \sum_i \left( d_i^{(r)} - o_i^{(r)} \right)^2.$$

4. Compute output-layer error signals:

$$\delta_i = (d_i - o_i) f'(\text{net}_i).$$

5. Compute hidden-layer error signals:

$$\delta_j = f'(\text{net}_j) \sum_i \delta_i w_{ji}.$$

6. Update output-layer weights

$$w_{ji} \leftarrow w_{ji} + \eta \delta_i o_j.$$

7. Update hidden-layer weights:

$$w_{kj} \leftarrow w_{kj} + \eta \delta_j o_k.$$

8. Repeat until the total loss is sufficiently small, or until a maximum number of epochs is reached.

### Batch vs Stochastic gradient descent

In batch learning, weights are updated after computing the total loss over all training examples. In stochastic learning, weights are updated after each training example.

Stochastic learning is usually preferred because it is often faster and may find better solutions by escaping some local optima.

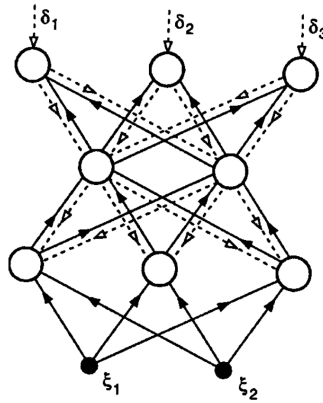


Figure 8: Backpropagation idea. Solid lines represent forward propagation; dashed lines denote the backward propagation of error.

## 2.7 Momentum and Learning Rate

### 2.7.1 Momentum

Momentum modifies the weight update by adding part of the previous update:

$$\Delta w(t) = \eta \delta_q o_p + \alpha \Delta w(t-1),$$

where  $\alpha \in [0, 1)$  is the momentum parameter. Then

$$w(t+1) = w(t) + \Delta w(t).$$

Momentum helps smooth learning. It can speed up movement in consistent descent directions and reduce oscillations.

### Learning rate

The learning rate  $\eta$  controls how large each update step is.

- If  $\eta$  is too small, then training loss convergence is slow.
- If  $\eta$  is too large, then the algorithm may overshoot minima or the loss may diverge.

Choosing  $\eta$  is therefore important in practice.

## 2.8 Overfitting and Underfitting

### Underfitting

Underfitting happens when the network is too simple to capture the input-output mapping. This can happen when there are too few hidden layers or too few hidden neurons.

Signals:

- high training loss;
- poor training accuracy;
- poor test accuracy.

### Overfitting

Overfitting happens when the network is too complex and learns the training examples too specifically. This can happen when there are too many hidden layers, too many hidden neurons, or too many parameters relative to the amount of data.

Signals:

- low training loss;
- high training accuracy;
- poor test accuracy.

The network memorises training data but generalises poorly to unseen examples.

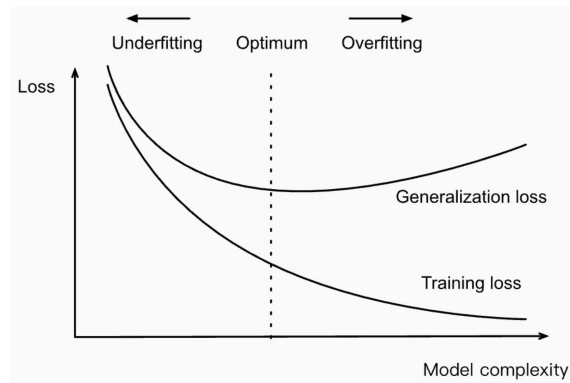


Figure 9: Model complexity curve.

### Preventing overfitting

Common strategies include:

- Use a validation set to monitor generalisation performance.

- Stop training early when validation performance stops improving.
- Reduce network size by using fewer hidden neurons or fewer hidden layers.
- Use regularisation to penalise overly large weights.
- Get more training data if possible.

## 2.9 Capabilities and Limitations of Multilayer Neural Networks

### Capabilities

Multilayer neural networks can model nonlinear decision boundaries. Unlike a single perceptron, a MLP can solve XOR and other nonlinearly separable problems.

With enough hidden neurons, multilayer networks can approximate very complex functions. This is the intuition behind the universal approximation result: neural networks with hidden layers can approximate a wide class of functions.

### Limitations

Backpropagation is based on gradient descent, so it may find a local minimum rather than the global minimum.

Training can be sensitive to:

- initial parameters;
- learning rate;
- network architecture;
- number of hidden neurons.

Neural networks can overfit if they have too many parameters. They can also require many training examples and significant computation. Their learned weights are often difficult to interpret directly.